Desafio Técnico, QA Lead

Nebula Forge Studios

Candidato: Thales Fernandes, QA Lead, INO Games

Contexto do Cenário

A Nebula Forge Studios apresenta um padrão crítico de detecção tardia de bugs, com 40% dos problemas identificados próximos ao release e 9 bugs críticos escapando para produção no último ciclo. Com apenas 2 QAs juniores, sem regressão estruturada e testes majoritariamente exploratórios e reativos, o desafio é elevar a qualidade sem comprometer a velocidade de entrega.

Dados dos últimos 2 meses

- 28 bugs críticos encontrados após freeze
- 9 bugs críticos escaparam para produção no último release
- 40% dos bugs encontrados tardiamente (próximo ao release)
- Tempo médio de correção: 3 dias

1. Diagnóstico e Prioridades (2 semanas)

Os 3 principais problemas

Problema 1, Ausência de regressão estruturada

Sem uma suíte mínima, cada build depende de memória e sorte. Isso explica diretamente os 9 bugs que escaparam para produção. Não há linha de base para comparar comportamentos entre versões.

Problema 2, Detecção tardia (shift-right)

40% dos bugs encontrados perto do release indica que os testes só acontecem no final do ciclo, QA como 'portão', não como parceiro. O custo de correção explode quanto mais tarde o bug é encontrado.

Problema 3, Tempo de correção alto (3 dias)

Bugs críticos encontrados tarde com ciclo de correção lento geram risco real em cada release, pressionando o time e aumentando a chance de hotfixes emergenciais pós-produção.

3 ações prioritárias para as próximas 2 semanas

#	Ação	Justificativa
1	Criar Smoke Test mínimo por feature nova	Bloqueia bugs óbvios antes da regressão. Custo baixo, retorno imediato.
2	Mapear fluxos críticos (progressão, matchmaking, economia)	Sem mapeamento não há regressão possível. É o alicerce de tudo.
3	Estabelecer critério de entrada em QA (DoR)	Evita que QA receba features incompletas e desperdice ciclos.

Nota: automação não entra agora porque sem mapeamento de fluxos e sem DoR, automatizar seria construir em areia.

2. Estrutura de Testes

Smoke Test

- Executado a cada build semanal, antes de qualquer teste mais profundo
- Cobre fluxos vitais: login, criação de partida, progressão básica, economia de itens
- Duração máxima: 30 a 45 min por QA
- Critério de saída: se o smoke falhar, a build é devolvida sem entrar em regressão

Regressão

- Baseada nos fluxos críticos mapeados
- Executada integralmente antes de cada release trimestral
- Versão parcial (smoke expandido) nas builds semanais, cobrindo as áreas tocadas na sprint
- Começa pequena e cresce organicamente com o produto

Testes Exploratórios

- Reservados para features novas e áreas de alto risco identificadas no planejamento
- Sessões com time-box (1h por sessão, com charter definido)
- Documentados com anotações leves, sem burocracia no início

O que NÃO testaria agora

- Automação de ponta a ponta (sem base de fluxos estável, vira dívida técnica)
- Performance e stress testing (prioridade futura)
- Localização e acessibilidade (relevante, mas não é o gargalo atual)
- Plataformas secundárias e dispositivos de baixa prioridade

3. Mudança no Fluxo para Antecipar Bugs

O problema central é que o time de QA entra depois que a feature está pronta. A mudança é fazer o time de QA entrar antes.

Mudanças práticas

- QA participa do refinamento/planejamento das features: lê o design doc, faz perguntas de borda, levanta riscos antes do desenvolvimento começar
- Testes de aceitação escritos antes do desenvolvimento (shift-left leve): o dev sabe exatamente o que vai ser validado, reduzindo mal-entendidos e bugs por interpretação
- Check de QA em PRs de features críticas: não é code review técnico, é validação de comportamento antes de entrar na build principal
- Bug triage semanal: revisar bugs abertos com o dev responsável toda semana, evitando que bugs 'durmam' e explodam no freeze

Como lidar com a resistência do time de dev?

A abordagem é mostrar que isso reduz interrupções deles, menos bug fix urgente perto do release. Não aumenta a carga, redistribui o risco ao longo do ciclo.

4. Alocação do QA no Ciclo de Produção

Fase	Presença de QA	Responsabilidades
Planejamento / Refinamento	**Ativa**	Levantar riscos, escrever critérios de aceite, mapear dependências
Desenvolvimento	**Pontual**	Tirar dúvidas de comportamento, revisar flows em progresso
Build semanal interna	**Ativa**	Smoke test, testes exploratórios de features novas
Pré-freeze	**Intensa**	Regressão completa, priorização de bugs, comunicação de risco
Freeze / Release	**Crítica**	Validação de fixes, go/no-go, acompanhamento de métricas
Pós-release	**Analítica**	Review de bugs escapados, atualização da suíte, retrospectiva

5. Delegação, O que Cada Um Faz

QA Juniores

- Execução dos smoke tests e regressões (seguindo scripts e checklists criados pelo Lead)
- Registro e documentação de bugs com template padronizado
- Testes exploratórios de features de menor risco
- Manutenção e atualização da suíte de regressão ao longo do tempo

QA Lead (eu)

- Mapear os fluxos críticos e criar a base da suíte de regressão
- Definir critérios de entrada e saída de QA (DoR / DoD)
- Comunicar riscos e decisões de release para stakeholders
- Triagem e priorização de bugs críticos
- Conduzir testes exploratórios em áreas de alto risco e features novas complexas
- Mentoria, revisão do trabalho dos juniores e condução do QA Guild
- Construir o relacionamento com o time de dev, esse é o trabalho mais estratégico no início

6. Iniciativa de Capacitação, QA Guild (8 semanas)

Um programa leve de capacitação e cultura de qualidade, rodando em paralelo com o trabalho do dia a dia:

Semanas	Foco
1–2	Onboarding de qualidade: mapeamento de fluxos críticos juntos, entendimento do produto
3–4	Técnicas de teste: como escrever bons casos de teste, exploratório com charter
5–6	Como reportar bugs que geram ação: título, severidade, passos, evidências
7–8	Revisão do ciclo + apresentação dos juniores sobre área que dominaram

Complementos do programa

- Sessão quinzenal de review de bugs escapados: 'o que poderíamos ter pego antes?', sem julgamento
- Pair testing ocasional: QA Lead + Junior juntos em features complexas
- Bug da semana: analisar um bug interessante em grupo, criando cultura de aprendizado contínuo

Objetivo: não é só melhorar habilidades técnicas, é criar senso de ownership de qualidade no time.

7. Decisão de Release, Bug Crítico no Freeze

Análise da situação

- Bug crítico identificado durante o freeze
- Afeta aproximadamente 20% dos jogadores
- Tempo de correção estimado: 2 dias
- Release planejado para amanhã

Decisão: recomendar adiamento de 48h

Um bug crítico afetando 1 em cada 5 jogadores não é aceitável em um jogo com foco em retenção. O dano à experiência do jogador e à reputação do estúdio supera o custo de atrasar 48h.

Comunicação da decisão

> *"Identificamos um bug crítico durante o freeze que afeta aproximadamente 20% da base de jogadores. A correção está estimada em 2 dias. Minha recomendação é adiar o release por 48h para garantir que o produto chegue aos jogadores no nível de qualidade esperado. Lançar agora representa risco concreto de impacto negativo na retenção e possível necessidade de hotfix emergencial, que tem custo operacional maior do que o adiamento. Estou disponível para alinhar o novo cronograma com o time."*

Se a decisão de negócio for lançar mesmo assim

- Documento formalmente o risco e a minha recomendação (registro escrito)
- Proponho plano de hotfix imediato já preparado para execução no dia seguinte
- Monitoro métricas de impacto nas primeiras horas pós-release
- Não bloqueio unilateralmente, mas a posição de QA fica registrada com clareza

Proposta: Feature Flags como mecanismo de mitigação de risco

Implementar feature flags no fluxo de desenvolvimento permite que novas funcionalidades sejam entregues na build sem ficarem imediatamente expostas a todos os jogadores. Em caso de bug crítico identificado após o release, a feature pode ser desativada remotamente em segundos, sem necessidade de hotfix, novo build ou nova versão publicada.

Como funcionaria na prática:

- Toda feature nova entra na build protegida por uma flag desativada por padrão
- QA testa com a flag ativa em ambiente controlado antes do release
- No release, a flag é ativada gradualmente (ex: 10% dos jogadores → 50% → 100%)
- Se um bug crítico for detectado pós-release, a flag é desligada instantaneamente, revertendo o comportamento sem downtime

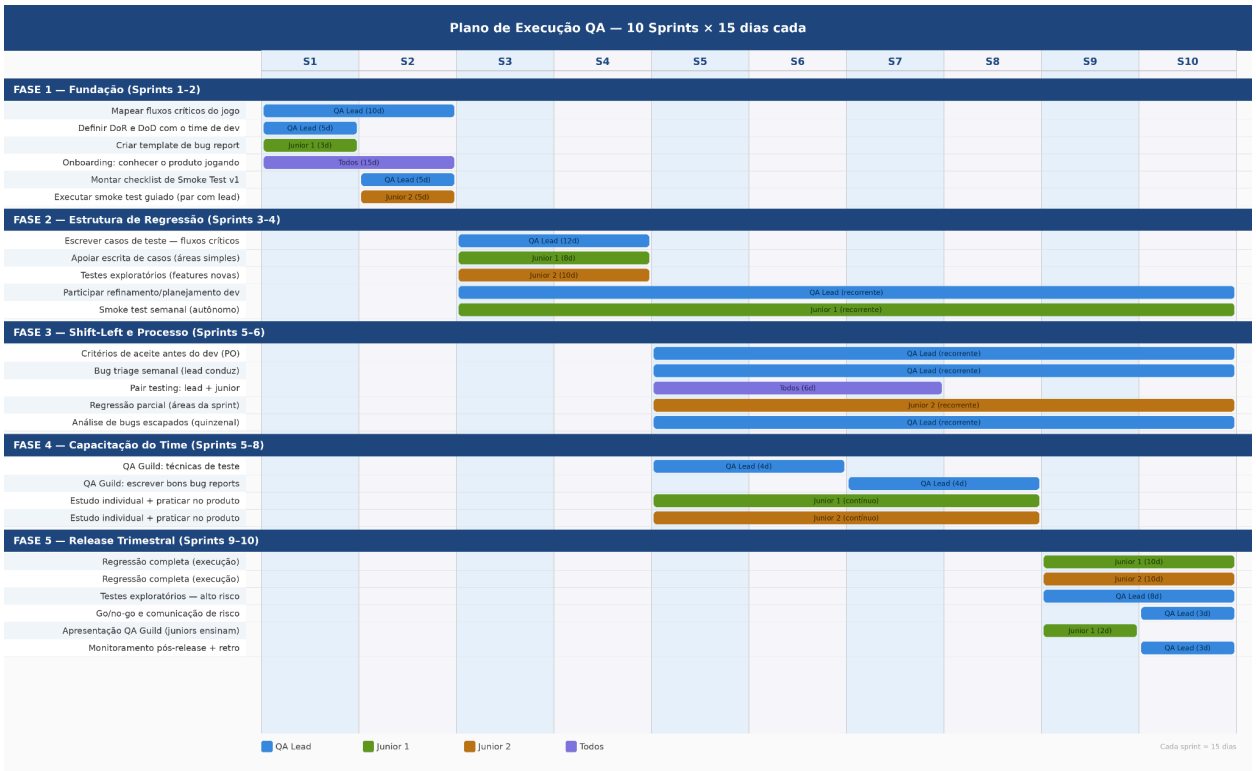
Benefícios diretos para o contexto do estúdio:

- Reduz drasticamente o impacto de bugs que escapam para produção
- Elimina a pressão de bloquear releases por bugs em features isoladas
- Permite rollout gradual, facilitando a detecção precoce de problemas em produção

Pré-requisito: requer alinhamento com o time de desenvolvimento para adoção, mas é exatamente o tipo de proposta que o time tende a abraçar, pois reduz a pressão sobre eles também em situações de crise.

Gráfico de Gantt, Plano de Execução QA

10 sprints de 15 dias cada. Distribuição de tarefas por nível e responsabilidade.



Legenda

- QA Lead (azul), tarefas estratégicas, decisão, mentoria e mão na massa em áreas críticas
- Junior 1 (verde), smoke tests, casos de teste simples, regressão
- Junior 2 (laranja), testes exploratórios supervisionados, regressão parcial
- Todos (roxo), atividades conjuntas: onboarding, pair testing, QA Guild

Por onde eu começaria

Antes de propor qualquer processo, ferramenta ou plano, meus primeiros dias seriam dedicados inteiramente a ouvir. Não chego com respostas prontas, chego com perguntas.

O primeiro movimento seria sentar individualmente com cada pessoa do time: os dois QAs juniores, os desenvolvedores e, se possível, alguém de produto. O objetivo não é fazer uma reunião formal, é entender como o trabalho acontece de verdade, não como está documentado. Quero saber onde cada um sente mais dor, o que já foi tentado antes e não funcionou, e o que eles acham que precisaria mudar.

Com os juniores, quero entender o nível de autonomia que já têm, como eles priorizam o que testar e onde se sentem inseguros. Isso define o ponto de partida real da capacitação, não um nível assumido.

Com os desenvolvedores, o objetivo é diferente: construir abertura. Time resistente à mudança geralmente não é resistente a melhorias, é resistente a ser interrompido ou julgado. Mostrar que estou ali para resolver um problema que também afeta eles, e não para fiscalizar, é o que abre espaço para qualquer iniciativa futura.

Só depois dessas conversas eu olharia para os dados, bugs reportados, histórico de releases, tickets em aberto, para cruzar o que ouvi com o que os números mostram.

O plano de ação vem do dia 2 em diante. O dia 1 é para ganhar contexto e, principalmente, começar a ganhar confiança. Processo sem confiança não sobrevive à primeira sprint.

> *"Um bug crítico que afeta 20% dos jogadores não é um problema de QA, é um problema de receita e de reputação. Meu trabalho é fazer o time enxergar isso antes que aconteça, não depois."*